

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

AÇIK KAYNAK EDA ARAÇLARI KULLANILARAK 16-BİT BİR
İŞLEMCI ÇİP TASARIMININ GERÇEKLEŞTİRİLMESİ VE YAPAY ZEKA
DESTEKLİ YAKLAŞIMLARIN İNCELENMESİ

Elif CİHAT

Nursena ATEŞ

İsmailcan USLU

Dilara ALKANAT

Meryem Betül ÇELİK

OCAK - 2026

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Başlığı: Açık Kaynak EDA Araçları Kullanılarak 16-Bit Bir İşlemci Çip
Tasarımının Gerçekleştirilmesi ve Yapay Zekâ Destekli
Yaklaşımların İncelenmesi

Elif CİHAT, BİLGİSAYAR MÜHENDİSLİĞİ

Nursena ATEŞ, BİLGİSAYAR MÜHENDİSLİĞİ

Öğrenciler İsmailcan USLU, BİLGİSAYAR MÜHENDİSLİĞİ

Dilara ALKANAT, BİLGİSAYAR MÜHENDİSLİĞİ

Meryem Betül ÇELİK, BİLGİSAYAR MÜHENDİSLİĞİ

Proje Danışmanları: Latif AKÇAY, BİLGİSAYAR MÜHENDİSLİĞİ

Teslim Tarihi: 01/01/2026

BEYAN

Erzurum Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü bitirme projesi yazım kurallarına uygun olarak hazırladığımız “Açık Kaynak EDA Araçları Kullanılarak 16-Bit Bir İşlemci Çip Tasarımının Gerçekleştirilmesi ve Yapay Zekâ Destekli Yaklaşımların İncelenmesi”

başlıklı proje dokümanının içindeki bütün bilgilerin doğru olduğunu, bilgilerin üretilmesi ve sunulmasında bilimsel etik kurallarına uygun davrandığımızı, kullandığımız bütün kaynakları atıf yaparak belirttiğimizi, maddi ve manevi desteği olan tüm kurum/kuruluş ve kişileri belirttiğimizi, burada sunduğumuz veri ve bilgileri unvan almak amacıyla daha önce hiçbir şekilde kullanmadığımızı beyan ederim.

01.01.2026

Elif CİHAT

Nursena ATEŞ

İsmailcan USLU

Dilara ALKANAT

Meryem Betül ÇELİK

ÖNSÖZ

Yarı iletken teknolojilerinin hızla gelişmesiyle birlikte, çip tasarımı günümüz bilgisayar mühendisliği alanının en kritik ve stratejik çalışma konularından biri hâline gelmiştir. Ancak ASIC tasarım süreçleri, çoğunlukla ticari ve lisans maliyeti yüksek EDA araçları gerektirdiğinden, öğrenciler ve akademik çalışmalar için erişimi sınırlı bir alan olarak görülmektedir. Bu bitirme projesi, açık kaynaklı EDA araçları kullanılarak uçtan uca bir çip tasarım sürecinin gerçekleştirilebileceğini göstermek ve bu süreci uygulamalı olarak deneyimlemek amacıyla hazırlanmıştır.

Proje kapsamında, LibreLane açık kaynak ASIC tasarım akışı kullanılarak 16-bit bir işlemci mimarisinin RTL seviyesinden başlayıp fiziksel doğrulama aşamalarına kadar ilerleyen tam bir tasarım süreci ele alınmıştır. Çalışma sırasında karşılaşılan en önemli zorluklar; açık kaynak araçların öğrenilmesi, tasarım akışının doğru yapılandırılması ve zamanlama ile fiziksel doğrulama kısıtlarının sağlanması olmuştur. Bununla birlikte, tasarımın kapsamı bilinçli olarak sade tutulmuş, karmaşık mimarilerden kaçınılarak sürecin anlaşılabilir ve doğrulanabilir olması hedeflenmiştir.

Bu çalışmanın gerçekleştirilmesindeki temel motivasyon, açık kaynaklı tasarım araçlarının sunduğu imkânları derinlemesine öğrenmek, çip tasarımının teorik bilgilerle sınırlı kalmadığını göstermek ve yapay zekâ destekli yaklaşımların gelecekte ASIC tasarım süreçlerine nasıl entegre edilebileceğine dair bir bakış açısı kazanmaktır. Bu yönüyle proje, yalnızca mevcut bir tasarımın gerçekleştirilmesini değil, aynı zamanda ileride yapılabilecek daha gelişmiş çalışmalar için bir temel oluşturmayı amaçlamaktadır.

Bitirme projesi süresince bilgi ve deneyimlerini paylaşarak çalışmanın her aşamasında yönlendirmelerde bulunan danışman hocama teşekkür ederim. Ayrıca, açık kaynak topluluklarının geliştirdiği araçlar ve sağladıkları dokümantasyonlar bu çalışmanın gerçekleştirilmesinde önemli katkı sağlamıştır.

Elif CİHAT
Nursena ATEŞ
İsmailcan USLU
Dilara ALKANAT
Meryem Betül ÇELİK
ERZURUM, 2026

İÇİNDEKİLER

Sayfa

Önsöz.....	IV
İçindekiler.....	V
Özet.....	VII
Şekiller Listesi.....	VIII
Simgeler ve Kısaltmalar.....	IX
1. GİRİŞ.....	1
2. LİTERATÜR TARAMASI.....	2
2.1 Açık Kaynak EDA'nın Ortaya Çıkışı ve Sektörel Önemi.....	2
2.2 Açık Kaynak RTL'den GDSII'ye Akışlar: LibreLane, OpenLane ve OpenROAD.....	3
2.3 SKY130 Açık PDK ve Fiziksel Doğrulama Ekosistemi.....	4
2.4 Yapay Zekâ Destekli ASIC Tasarım Yaklaşımları.....	6
2.5 Literatürdeki Boşluk ve Bu Çalışmanın Konumlandırılması.....	7
3. TASARIM YÖNTEMİ VE KULLANILAN ARAÇLAR.....	8
3.1 Açık Kaynak ASIC Tasarım Yaklaşımı.....	8
3.2 LibreLane Tasarım Akışı.....	8
3.3 Kullanılan Yazılım Araçları.....	8
3.3.1 Icarus Verilog.....	8
3.3.2 GTKWave.....	8
3.3.3 Yosys.....	9
3.3.4 OpenROAD.....	9
3.3.5 Magic ve KLayout.....	9
3.3.6 SkyWater 130 nm (sky130A) PDK.....	9
3.3.7 nix-shell (Nix Package Manager).....	9
3.3.8 Visual Studio Code (VS Code).....	9
3.3.9 Neovim.....	9
3.4 Yapay Zekâ Destekli Yaklaşımların Değerlendirilmesi.....	9
4. MATERYAL VE METOT.....	10
4.1 Kullanılan Materyaller.....	10
4.1.1 Donanım ve İşletim Sistemleri.....	10
4.2 Kullanılan Yöntemler.....	10
4.2.1 Tasarım Yaklaşımı.....	11
4.2.2 16-bit CPU Mimarisi.....	14
4.2.3 RTL Simülasyon ve Doğrulama.....	15
4.2.4 Fiziksel Tasarım Akışı.....	17
4.3 Değerlendirme.....	17
5. BULGULAR VE TARTIŞMA.....	18
5.1 Sentez Sonuçlarının Değerlendirilmesi.....	18
5.2 Zamanlama Analizi (Static Timing Analysis) Bulguları.....	18
5.3 Yerleştirme ve Yönlendirme (Place & Route) Sonuçları.....	18
5.4 Fiziksel Doğrulama Bulguları (DRC, LVS ve Antenna Kontrolleri).....	19

5.5 Genel Değerlendirme.....	19
6. GELECEK DÖNEM ÇALIŞMALARI.....	20
6.1 Yapay Zeka Destekli Donanım Doğrulama Yaklaşımı.....	20
6.2 Yapay Zekâ Entegrasyonu İçin Ara Katman Yazılım Mimarisi.....	20
6.3 Otomatik Hata Analizi ve Akıllı Geri Bildirim Mekanizması.....	21
7. SONUÇLAR.....	21
Kaynaklar.....	23

ÖZET

Açık Kaynak EDA Araçları Kullanılarak 16-Bit Bir İşlemci Çip Tasarımının Gerçekleştirilmesi ve Yapay Zekâ Destekli Yaklaşımların İncelenmesi

Elif CİHAT
Nursena ATEŞ
İsmailcan USLU
Dilara ALKANAT
Meryem Betül ÇELİK
ERZURUM TEKNİK ÜNİVERSİTESİ
Bilgisayar Mühendisliği Bölümü

Bu bitirme projesinde, açık kaynaklı Elektronik Tasarım Otomasyonu (EDA) araçları kullanılarak 16-bit bir işlemci çipinin uçtan uca ASIC tasarım süreci ele alınmıştır. Çalışmanın temel amacı, ticari araçlara ihtiyaç duyulmadan LibreLane açık kaynak tasarım akışı ile gerçekçi bir çip tasarımının gerçekleştirilebileceğini göstermek ve bu süreci uygulamalı olarak deneyimlemektir. Ayrıca, yapay zekâ destekli yaklaşımların ASIC tasarım süreçlerine entegrasyon potansiyeli incelenmiştir.

Projenin hipotezi, açık kaynaklı EDA araçlarının akademik ve eğitim amaçlı çip tasarımı için yeterli doğruluk ve üretilebilirlik sağlayabildiği yönündedir. Çalışma kapsamında tek bir örnek tasarım olarak 16-bit veri genişliğine sahip, senkron çalışan bir işlemci sistemi geliştirilmiş ve bu tasarım RTL seviyesinden fiziksel doğrulama aşamasına kadar ilerletilmiştir. Örneklem olarak tasarlanan işlemci sistemi, işlemci çekirdeği, bellek birimleri ve kontrol mantığından oluşmaktadır.

Yöntem olarak Verilog dili ile RTL tasarımı gerçekleştirilmiş, fonksiyonel doğrulama testbench ve simülasyonlar aracılığıyla yapılmıştır. Sentez işlemleri Yosys kullanılarak SkyWater 130 nm standart hücre kütüphanesine eşlenmiş, yerleştirme ve yönlendirme adımları OpenROAD altyapısı ile tamamlanmıştır. Fiziksel doğrulama kapsamında DRC, LVS ve antenna kontrolleri uygulanmıştır.

Elde edilen bulgular, tasarımın belirlenen zamanlama kısıtlarını sağladığını ve Sky130 üretim süreci kurallarına uygun olduğunu göstermektedir. Sonuç olarak bu çalışma, açık kaynaklı EDA araçlarının ASIC tasarımında etkin bir şekilde kullanılabileceğini ve yapay zekâ destekli yöntemlerin gelecekte tasarım süreçlerinin iyileştirilmesi için önemli bir potansiyel sunduğunu ortaya koymaktadır.

Anahtar Kelimeler: ASIC tasarımı, açık kaynak EDA, LibreLane, Sky130, çip tasarımı, yapay zekâ

ŞEKİLLER LİSTESİ

	Sayfa
Şekil 2.1. Sky130 PDK hedeflenerek hazırlanan 16-bit işlemci LibreLane konfigürasyon ayarları	4
Şekil 2.2. 16-bit işlemci sistemi için 100 MHz saat kısıtının LibreLane üzerinde tanımlanması	4
Şekil 2.3. Sky130 PDK kapsamında kullanılan sky130_fd_sc_hd standart hücre kütüphanesi	4
Şekil 2.4. Simple16_system tasarımı için sentez (synthesis) kısıtlarının tanımlanması	5
Şekil 2.5. Simple16_system tasarımı için yonga (die) ve çekirdek (core) alan boyutlandırmasının tanımlanması	5
Şekil 2.6. Simple16_system tasarımı için floorplanning (çip yerleşim planı) ayarlarının tanımlanması	6
Şekil 2.7. Simple16_system tasarımı için yerleştirme (placement) kısıtlarının tanımlanması	6
Şekil 2.8. Simple16_system tasarımı için saat ağacı sentezi (Clock Tree Synthesis – CTS) ayarlarının tanımlanması	7
Şekil 2.9. simple16_system tasarımı için yönlendirme (routing) kısıtlarının tanımlanması	7
Şekil 2.10. simple16_system tasarımı için tahmini donanım kaynakları ve performans hedefleri	8
Şekil 3.1. simple16_system tasarımının fonksiyonel doğrulaması için oluşturulan testbench (simple16_system_tb) kodu	8
Şekil 3.2. Testbench içerisinde 100 MHz saat sinyalinin (clock) üretilmesi	8
Şekil 3.3. Testbench içerisinde reset, test senaryosu ve doğrulama kontrolünün tanımlanması	9
Şekil 3.4. ALU modülüne ait sinyallerin GTKWave üzerinde simülasyon dalga formu ile doğrulanması	9
Şekil 3.5. Veri belleği (data memory) modülüne ait adres, veri ve yazma kontrol sinyallerinin GTKWave üzerinde simülasyon dalga formu ile incelenmesi	9
Şekil 3.6. Komut belleği (instruction memory) modülüne ait opcode ve veri sinyallerinin GTKWave üzerinde simülasyon dalga formu ile doğrulanması	10
Şekil 3.7. Testbench simülasyonu sonucunda sistemin doğru şekilde çalıştığını gösteren konsol çıktısı	10

SİMGELER VE KISALTMALAR

Simgeler

clk : Saat sinyali
rst : Reset sinyali
acc_out : İşlemci akümülatör çıkışı
halted : İşlemcinin durduğunu gösteren kontrol sinyali

Kısaltmalar

ASIC : Application Specific Integrated Circuit
EDA : Electronic Design Automation
RTL : Register Transfer Level
PNR : Place and Route
DRC : Design Rule Check
LVS : Layout Versus Schematic
STA : Static Timing Analysis
GDS : Graphic Data System
SDC : Synopsys Design Constraints

1. GİRİŞ

Yarı iletken teknolojilerindeki hızlı gelişim, çip tasarımı bilgisayar mühendisliğinin en önemli ve stratejik alanlarından biri hâline getirmiştir. Günümüzde işlemci, denetleyici ve özel amaçlı entegre devreler (ASIC), performans, güç verimliliği ve güvenlik gereksinimlerini karşılamak amacıyla yaygın olarak kullanılmaktadır. Ancak bu tasarımların gerçekleştirilmesi çoğu zaman yüksek maliyetli ticari Elektronik Tasarım Otomasyonu (EDA) araçlarına bağımlı olduğundan, akademik çalışmalar ve öğrenci projeleri için erişimi sınırlı bir alan oluşturmaktadır.

Son yıllarda açık kaynaklı donanım tasarımı ve EDA araçlarının gelişmesi, bu alanda önemli bir dönüşüm yaratmıştır. Açık kaynaklı üretim süreçleri ve tasarım akışları sayesinde, gerçekçi ve üretilebilir çip tasarımlarının akademik ortamda gerçekleştirilmesi mümkün hâle gelmiştir. Özellikle SkyWater 130 nm üretim süreci ve bu süreci hedefleyen açık kaynaklı tasarım akışları, ASIC tasarımının teorik bilgilerin ötesine geçerek uygulamalı olarak öğrenilmesine olanak tanımaktadır.

Bu bitirme projesi, açık kaynaklı LibreLane ASIC tasarım akışı kullanılarak 16-bit bir işlemci çipinin uçtan uca tasarım sürecini ele almaktadır. Çalışmada, RTL tasarımından başlayarak sentez, yerleştirme ve yönlendirme, zamanlama analizi ve fiziksel doğrulama aşamaları bütüncül bir yaklaşımla incelenmiştir. Ayrıca, yapay zekâ destekli yaklaşımların çip tasarım süreçlerinde analiz ve iyileştirme amacıyla kullanılabilirliği değerlendirilerek, gelecekteki çalışmalar için bir temel oluşturulması hedeflenmiştir.

Bu kapsamda gerçekleştirilen çalışma, açık kaynaklı EDA araçlarının eğitim ve araştırma amaçlı çip tasarımı için uygulanabilirliğini ortaya koymayı ve ASIC tasarım sürecine dair sistematik bir bakış açısı sunmayı amaçlamaktadır.

2. LİTERATÜR TARAMASI

Bu bölümde, açık kaynaklı Elektronik Tasarım Otomasyonu (EDA) araçlarının ortaya çıkışı, gelişimi ve ASIC tasarım süreçlerindeki yeri literatürdeki çalışmalar ışığında incelenmektedir. Öncelikle açık kaynak EDA yaklaşımlarının sektörel ve akademik önemi ele alınmış, ardından RTL'den GDSII'ye uzanan açık kaynak tasarım akışları ve bu akışlarda kullanılan araçlar değerlendirilmiştir. Ayrıca, SKY130 açık PDK ekosistemi ve fiziksel doğrulama süreçleri literatür çerçevesinde açıklanmıştır. Son olarak, yapay zekâ destekli ASIC tasarım yaklaşımlarına ilişkin güncel çalışmalar incelenerek bu çalışmanın literatürdeki konumu ortaya konulmuştur.

2.1 Açık Kaynak EDA'nın Ortaya Çıkışı ve Sektörel Önemi

ASIC tasarım süreçleri; RTL tasarımı, mantıksal sentez, yerleştirme-yönlendirme, zamanlama analizi ve fiziksel doğrulama gibi ardışık aşamaları içeren karmaşık bir Elektronik Tasarım Otomasyonu (EDA) zinciri gerektirir. Bu zincirin uzun yıllar boyunca büyük ölçüde kapalı kaynaklı, lisans maliyeti yüksek ve kurumsal erişim gerektiren araçlara bağımlı olması, çip tasarımını belirli şirketlerin hâkim olduğu bir ekosisteme dönüştürmüştür. Nitekim EDA yazılım pazarında sınırlı sayıdaki tedarikçinin baskın konumda olduğu ve bu şirketlerin bazı pazarlarda yüksek toplam paylara ulaştığı; ayrıca EDA araçlarına yönelik ihracat kısıtlamalarının sektörel etkilerinin tartışıldığı görülmektedir [1], [2].

Açık kaynak EDA yaklaşımlarının önemi yalnızca maliyet avantajı ile sınırlı değildir. EDA sektörünün yoğunlaşmış yapısı, akademik ve endüstriyel tasarım süreçlerini ekonomik olduğu kadar politik ve stratejik açıdan da bağımlı hâle getirmektedir. Özellikle çip tasarım yazılımlarına yönelik ihracat kısıtlamaları ve lisans sınırlamaları, donanım tasarımında araç bağımsızlığının kritik bir gereklilik olduğunu ortaya koymuştur [1], [2]. Bu bağlamda açık kaynak EDA araçları, yalnızca düşük maliyetli alternatifler değil; aynı zamanda bilgiye erişimin demokratikleşmesini, akademik özgürlüğü ve teknolojik bağımsızlığı destekleyen stratejik bir dönüşüm olarak değerlendirilmektedir.

Bu yoğunlaşmış yapı; lisans maliyetleri, eğitim ve öğrenme bariyerleri ile yüksek geçiş maliyetleri gibi nedenlerle özellikle akademik çalışmalar ve küçük ekipler açısından önemli bir erişim engeli oluşturmaktadır. Dolayısıyla çip tasarımının yalnızca teorik düzeyde kalmayıp uçtan uca deneyimlenmesi çoğu zaman kapalı ekosistemlere bağımlı hâle gelmektedir. Açık kaynak EDA yaklaşımı bu noktada yalnızca “ücretsiz yazılım” anlamına gelmemekte; tasarım akışının şeffaflaşması, tekrarlanabilir deney ortamlarının oluşturulması ve öğrenme topluluklarının

büyümesi yoluyla donanım tasarımının daha geniş kitlelere yayılmasını hedefleyen bütüncül bir yaklaşım sunmaktadır [2].

Açık kaynak EDA'nın etkisini güçlendiren en kritik unsurlardan biri, üretim hedefli fiziksel tasarım için gereken süreç bilgilerinin (Process Design Kit – PDK) erişilebilir hâle gelmesidir. Bu bağlamda SKY130 gibi foundry destekli açık PDK girişimleri, fiziksel doğrulama (DRC/LVS) ve standart hücre kütüphaneleri gibi bileşenlerin daha geniş kullanıcı kitleleri tarafından kullanılabilmesini mümkün kılmıştır [10], [11]. SKY130'un açık biçimde yayımlanması, açık kaynak akışların yalnızca eğitim amaçlı prototipler üretmekle sınırlı kalmayıp üretim kurallarına dayanan daha gerçekçi tasarım hedeflerine yaklaşmasına katkı sağlamıştır.

Buna ek olarak açık kaynak ekosistemde, araçların tekil kullanımından ziyade entegrasyon ve uçtan uca akış yaklaşımının ön plana çıktığı görülmektedir. Çünkü ASIC tasarımında başarı; sentez, fiziksel tasarım ve doğrulama adımlarının birlikte, tutarlı kısıtlarla ve tekrarlanabilir biçimde yürütülmesine bağlıdır. Bu nedenle literatürde OpenROAD ve OpenLane gibi girişimler, açık kaynak EDA'yı pratikte uygulanabilir kılmak için uçtan uca otomasyonu hedefleyen önemli yapı taşları olarak konumlandırılmaktadır [2], [4].

Bu çalışmanın motivasyonu, literatürde vurgulanan bu dönüşümle doğrudan örtüşmektedir. Açık kaynak EDA araçları kullanılarak SKY130 hedefli bir tasarımın RTL'den fiziksel doğrulama adımlarına kadar ilerletilebildiğini uygulamalı biçimde göstermek ve bu süreçte kullanılan araç ve akış bileşenlerini sistematik olarak değerlendirmek amaçlanmaktadır.

2.2 Açık Kaynak RTL'den GDSII'ye Akışlar: LibreLane, OpenLane ve OpenROAD

ASIC tasarımında başarı, RTL seviyesinde doğrulanmış bir tasarımın fiziksel üretime uygun bir yerleşime (GDSII) dönüştürülmesini sağlayan uçtan uca ve tutarlı bir akışın kurulmasına bağlıdır. Literatürde açık kaynak EDA araçlarının bu gereksinimi karşılayabilmesi için tekil araç kullanımından ziyade, sentezden fiziksel doğrulamaya kadar olan adımların otomasyon ve entegrasyon ilkeleriyle birleştirildiği akışların geliştirilmesine odaklanıldığı görülmektedir [3], [5].

Bu bağlamda öne çıkan çalışmalardan biri OpenROAD projesidir. OpenROAD, dijital yerleştirme ve yönlendirme süreçlerini açık kaynak olarak ele almayı ve bu süreçleri minimum insan müdahalesiyle gerçekleştirebilecek bir altyapı sunmayı hedeflemektedir. Ajayı ve arkadaşları, OpenROAD'un temel vizyonunu “kendini yöneten” (self-driving) fiziksel tasarım yaklaşımı olarak tanımlamış ve klasik EDA akışlarında yoğun emek gerektiren arka uç aşamalarının otomasyonla sadeleştirilebileceğini ortaya koymuştur [3], [6].

OpenROAD altyapısının pratikte kullanılabilir hâle gelmesini sağlayan önemli bir bileşen ise OpenLane akışıdır. OpenLane, RTL'den GDSII'ye kadar uzanan tasarım sürecini; sentez (Yosys), yerleştirme-yönlendirme (OpenROAD) ve fiziksel doğrulama (Magic/KLayout) adımlarını tek bir otomatik akış içerisinde birleştirmektedir. Literatürde OpenLane, özellikle SKY130 açık PDK ile birlikte kullanıldığında akademik ve prototipleme amaçlı ASIC tasarımlarında tekrarlanabilir ve üretim kurallarına dayalı sonuçlar elde edilebildiğini göstermesi açısından sıklıkla referans verilen bir yapı olarak öne çıkmaktadır [7].

OpenLane üzerine inşa edilen ve daha modüler bir yaklaşım sunan LibreLane ise açık kaynak EDA ekosisteminde akış yönetimini bir altyapı katmanı olarak ele almaktadır. LibreLane, farklı tasarım aşamalarını yapılandırılabilir konfigürasyonlar aracılığıyla yönetmeye imkân tanıyarak, tasarımcıların zamanlama kısıtları (SDC), yerleşim parametreleri (floorplan) ve saat ağacı sentezi (CTS) gibi kritik ayarları sistematik biçimde kontrol edebilmesini amaçlamaktadır [8]. Bu özellik, tasarım uzayı keşfi ve farklı konfigürasyonların karşılaştırılması açısından önemli bir avantaj sunmaktadır.

Literatürde vurgulanan ortak nokta; açık kaynak RTL'den GDSII'ye akışların başarısının, kullanılan araçların tek tek gücünden ziyade araçlar arasındaki entegrasyonun tutarlılığına ve kısıtların doğru modellenmesine bağlı olduğudur [5], [7]. Bu nedenle açık kaynak akışların etkin kullanımı, tasarımcının akış parametrelerini bilinçli şekilde yönetmesini gerektirmektedir.

Bu çalışma kapsamında tercih edilen LibreLane tabanlı yaklaşım, literatürde tanımlanan bu prensiplerle uyumlu olacak biçimde, RTL tasarımından başlayarak sentez, yerleştirme-yönlendirme ve fiziksel doğrulama adımlarının tek bir bütüncül akış içerisinde ele alınmasına olanak sağlamaktadır. Bu yaklaşımın akademik ortamda uygulanabilirliği, RISC-V tabanlı bir System-on-Chip tasarımının OpenLane akışı kullanılarak başarıyla gerçekleştirildiği güncel çalışmalarla da gösterilmiştir [18].

2.3 SKY130 Açık PDK ve Fiziksel Doğrulama Ekosistemi

ASIC tasarımında “üretilebilirlik” kavramı, yalnızca RTL seviyesinde doğru çalışan bir tasarım elde edilmesini değil; aynı zamanda tasarımın hedeflenen üretim sürecinin tüm fiziksel kurallarına uygun olmasını da gerektirir. Bu nedenle fiziksel tasarım aşamasında gerçekleştirilen Design Rule Check (DRC), Layout Versus Schematic (LVS) ve antenna kontrolleri, uçtan uca ASIC akışının vazgeçilmez bileşenleri olarak kabul edilmektedir. Literatürde, açık kaynak EDA akışlarının gerçekçi biçimde değerlendirilebilmesi için bu doğrulama adımlarının mutlaka hedef

üretim sürecine dayalı bir Process Design Kit (PDK) ile yürütülmesi gerektiği vurgulanmaktadır [9].

Bu bağlamda SKY130 açık PDK, açık kaynak EDA ekosistemi açısından kritik bir dönüm noktasıdır. SKY130, foundry destekli ve açık biçimde yayımlanmış bir PDK olarak; standart hücre kütüphaneleri, tasarım kuralları, fiziksel doğrulama dosyaları ve süreç modelleri gibi üretim odaklı bileşenleri içermektedir [10]. Bu sayede açık kaynak EDA araçları kullanılarak geliştirilen tasarımlar, yalnızca teorik düzeyde değil, gerçek bir üretim sürecinin kısıtları altında değerlendirilebilmektedir. Literatürde SKY130'un yaygınlaşması, açık kaynak akışların "akademik prototip" sınırının ötesine geçmesine olanak tanıyan temel etkenlerden biri olarak gösterilmektedir [9], [10].

Açık PDK'lerin pratikte kullanılabilmesi, farklı EDA araçlarının beklediği dosya yapıları ve formatlar arasında uyum sağlanmasını gerektirir. Bu amaçla geliştirilen açık kaynak PDK entegrasyon araçları, SKY130 gibi PDK'lerin Magic, KLayout ve Netgen gibi açık kaynak doğrulama araçlarıyla uyumlu hâle getirilmesini sağlamaktadır [11]. Bu entegrasyon katmanı, tasarımcının fiziksel doğrulama sürecini manuel müdahaleye ihtiyaç duymadan, otomatik ve tekrarlanabilir biçimde yürütmesine imkân tanır.

Fiziksel doğrulama sürecinde DRC kontrolleri, yerleşim geometrisinin üretim kurallarına uygunluğunu denetlerken; LVS kontrolleri, elde edilen yerleşimin mantıksal tasarımla birebir uyumlu olup olmadığını ortaya koymaktadır. Antenna kontrolleri ise üretim sırasında oluşabilecek potansiyel yüklenme sorunlarını önceden tespit etmeyi amaçlamaktadır. Literatürde bu adımların başarıyla tamamlanması, bir ASIC tasarımının "üretime hazır" olarak nitelendirilebilmesi için temel kriterler arasında yer almaktadır [12].

Açık kaynak EDA ekosisteminin önemli kazanımlarından biri, bu doğrulama adımlarının ticari araçlara bağımlı olmaksızın gerçekleştirilebilmesidir. Magic ve KLayout gibi açık kaynak araçlar, SKY130 PDK ile birlikte kullanıldığında hem görsel inceleme hem de kural tabanlı doğrulama açısından yeterli bir altyapı sunmaktadır [11]. Bu durum, açık kaynak akışların yalnızca tasarım üretme değil, aynı zamanda tasarımın fiziksel doğruluğunu belgeleyebilme yeteneğine sahip olduğunu göstermektedir.

Bu çalışma kapsamında SKY130 açık PDK hedeflenerek yürütülen fiziksel tasarım süreci; yerleştirme ve yönlendirme adımlarının ardından DRC, LVS ve antenna kontrollerinin uygulanmasını içermektedir. Böylece literatürde tanımlanan üretilebilirlik odaklı açık kaynak akış yaklaşımıyla uyumlu biçimde, tasarımın hem işlevsel hem de fiziksel açıdan doğrulanması amaçlanmıştır. Bu yaklaşım, açık kaynak EDA araçlarının uygun PDK ve doğrulama altyapısı ile

birlikte kullanıldığında, akademik düzeyde güvenilir ve gerçekçi ASIC tasarımlarının elde edilebileceğini göstermektedir.

2.4 Yapay Zekâ Destekli ASIC Tasarım Yaklaşımları

ASIC tasarım süreçleri; yüksek boyutlu, çok kısıtlı ve çoğu zaman NP-zor olarak sınıflandırılan optimizasyon problemleri içermektedir. Özellikle yerleştirme (placement), yönlendirme (routing), zamanlama tahmini (timing prediction) ve güç/alan/performans (PPA) optimizasyonu gibi fiziksel tasarım aşamaları, tasarım kalitesini doğrudan etkileyen karmaşık karar uzaylarına sahiptir. Bu nedenle literatürde, klasik sezgisel (heuristic) yöntemlerin yanında yapay zekâ (AI) ve makine öğrenmesi (ML) tabanlı yaklaşımların EDA süreçlerine entegre edilmesine yönelik çalışmaların son yıllarda hız kazandığı görülmektedir [15].

Makine öğrenmesi tabanlı yöntemler, EDA bağlamında genellikle iki ana amaçla kullanılmaktadır: (i) tasarım metriklerinin (zamanlama, güç, alan) erken aşamada tahmin edilmesi ve (ii) optimizasyon kararlarının otomatik olarak iyileştirilmesi. Özellikle yerleşim ve floorplanning problemlerinde derin öğrenme ve pekiştirmeli öğrenme yaklaşımlarının, geleneksel yöntemlere kıyasla rekabetçi sonuçlar üretebildiği raporlanmıştır [14]. Bu çalışmalar, fiziksel tasarımın insan uzmanlığından bağımsız biçimde “öğrenen sistemler” tarafından yönlendirilebileceğini göstermesi açısından dikkat çekicidir.

Bununla birlikte literatürde önemli bir ayrım bulunmaktadır: AI tabanlı yöntemlerin büyük bir kısmı, tasarım akışının tek bir aşamasını (örneğin yalnızca floorplanning veya zamanlama tahmini) hedef almaktadır. Bu yaklaşımlar, belirli metriklerde iyileşme sağlasa da, uçtan uca ASIC akışının tamamına entegre edilmesi çoğu zaman sınırlı kalmaktadır [15]. Dolayısıyla, AI destekli yöntemlerin pratik değeri; yalnızca tekil optimizasyon başarısından ziyade, tasarım akışıyla nasıl ve ne ölçüde bütünleşebildiği üzerinden değerlendirilmektedir.

Son dönemde, büyük dil modelleri (LLM) gibi daha genel amaçlı AI sistemlerinin EDA süreçlerinde kullanımı da literatürde tartışılmaya başlanmıştır. Bu çalışmalar, LLM’lerin betik üretimi, akış konfigürasyonu, hata ayıklama ve tasarım karar destek mekanizmaları gibi alanlarda yardımcı araçlar olarak kullanılabileceğini öne sürmektedir [16]. Ancak bu yaklaşımın henüz erken aşamada olduğu; doğrulanabilirlik, güvenilirlik ve ölçülebilir performans kazanımları açısından ek araştırmalara ihtiyaç duyulduğu vurgulanmaktadır.

Açık kaynak RTL’den GDSII’ye akışlar, AI destekli yaklaşımlar için özellikle uygun bir deney ortamı sunmaktadır. Bunun temel nedeni, bu akışların tekrarlanabilir deney koşulları sağlaması ve farklı konfigürasyonların sistematik biçimde karşılaştırılmasına imkân tanınmasıdır. Literatürde,

açık kaynak akışların bu yönüyle AI tabanlı tasarım uzayı keşfi (design space exploration) ve optimizasyon çalışmalarına zemin oluşturabileceği ifade edilmektedir [13], [15].

Bu çalışma kapsamında yapay zekâ, doğrudan tasarım üreten bir bileşen olarak değil; tasarım sürecinin analiz edilmesi, karar noktalarının değerlendirilmesi ve gelecekteki iyileştirmeler için yol gösterici bir perspektif olarak ele alınmaktadır. Bu yaklaşım, literatürde açık kaynak EDA akışlarının mevcut olgunluk düzeyi ve AI entegrasyonuna ilişkin sınırlamalarla uyumlu olup, ileri çalışmalara yönelik gerçekçi bir çerçeve sunmayı amaçlamaktadır. Son dönemde, agent tabanlı yapay zekâ çerçeveleri kullanılarak ASIC tasarım akışlarının otomatikleştirilmesini hedefleyen yaklaşımlar da önerilmiştir [17].

2.5 Literatürdeki Boşluk ve Bu Çalışmanın Konumlandırılması

Literatür genel olarak değerlendirildiğinde, açık kaynak EDA araçlarının ve SKY130 gibi foundry destekli açık PDK girişimlerinin ASIC tasarımını akademik ve araştırma ortamları için daha erişilebilir hâle getirdiği görülmektedir. OpenROAD, OpenLane ve LibreLane gibi akışlar; RTL tasarımından fiziksel doğrulamaya kadar uzanan süreçte açık kaynak araçlarla üretim kurallarına dayalı sonuçlar elde edilebildiğini ortaya koymaktadır [5]–[10]. Bununla birlikte literatürde raporlanan çalışmaların önemli bir kısmı ya belirli bir tasarım aşamasına odaklanmakta ya da akışın yalnızca sınırlı bir bölümünü kapsamaktadır.

Özellikle eğitim ve akademik bağlamda, uçtan uca (RTL→GDSII) bir ASIC tasarım sürecinin; kısıt tanımları, zamanlama hedefleri ve fiziksel doğrulama adımlarıyla birlikte bütüncül biçimde ele alındığı çalışmaların sayısı görece sınırlıdır [7], [8]. Yapay zekâ destekli yaklaşımlar açısından bakıldığında ise, bu yöntemlerin açık kaynak uçtan uca akışlarla tam entegrasyonu ve eğitim senaryolarındaki uygulanabilirliği henüz gelişme aşamasındadır [13]–[16].

Bu bağlamda gerçekleştirilen çalışma, LibreLane tabanlı açık kaynak ASIC tasarım akışı kullanılarak SKY130 hedefli 16-bit bir işlemci tasarımının RTL'den fiziksel doğrulamaya kadar ilerletilmesini ele almakta ve süreci uygulamalı biçimde ortaya koymaktadır. Her ne kadar OpenLane kullanılarak RISC-V tabanlı SoC tasarımlarının gerçekleştirildiği çalışmalar bulunsada [18], bu çalışmaların çoğu belirli mimarilere odaklanmakta ve farklı konfigürasyonların tasarım çıktıları üzerindeki etkilerini sınırlı düzeyde ele almaktadır. Bu yönüyle çalışma, literatürdeki boşluğa katkı sağlamayı ve açık kaynak ASIC tasarım süreçlerinin eğitim ve araştırma bağlamındaki uygulanabilirliğini ortaya koymayı hedeflemektedir.

3. TASARIM YÖNTEMİ VE KULLANILAN ARAÇLAR

Bu bölümde bitirme projesi kapsamında izlenen tasarım yöntemi ve kullanılan yazılım araçları ayrıntılı olarak açıklanmaktadır. Çalışmada, açık kaynaklı ASIC tasarım akışı esas alınmış ve tasarım süreci RTL seviyesinden fiziksel doğrulama aşamasına kadar sistematik bir biçimde yürütülmüştür. Amaç, gerçekçi bir çip tasarım sürecini uygulamalı olarak deneyimlemek ve açık kaynaklı araçların bu süreçteki yeterliliğini değerlendirmektir.

3.1 Açık Kaynak ASIC Tasarım Yaklaşımı

Bu projede, ticari EDA araçlarına alternatif olarak açık kaynaklı tasarım araçları tercih edilmiştir. Açık kaynaklı araçların kullanımı, tasarım sürecinin şeffaflığını artırmakta ve akademik çalışmalar için erişilebilir bir ortam sunmaktadır. Tasarım akışı boyunca kullanılan araçlar, endüstride uygulanan klasik ASIC tasarım adımlarını temel almaktadır.

3.2 LibreLane Tasarım Akışı

LibreLane, uçtan uca bir ASIC tasarım sürecini otomatikleştiren açık kaynaklı bir tasarım akışıdır. Bu çalışma kapsamında LibreLane, RTL sentezi, yerleştirme ve yönlendirme, zamanlama analizi ve fiziksel doğrulama adımlarını kapsayacak şekilde kullanılmıştır. LibreLane'in modüler yapısı, tasarımın farklı aşamalarının ayrı ayrı incelenmesine ve kontrol edilmesine olanak sağlamıştır.

3.3 Kullanılan Yazılım Araçları

Bu çalışmada kullanılan başlıca yazılım araçları aşağıda özetlenmiştir.

3.3.1 Icarus Verilog

Icarus Verilog, Verilog dili ile geliştirilen RTL tasarımın derlenmesi ve simülasyonunun gerçekleştirilmesi amacıyla kullanılmıştır. Tasarlanan 16-bit işlemci sistemi için oluşturulan testbench dosyaları Icarus Verilog üzerinde çalıştırılmış ve sistemin farklı çalışma senaryoları altında doğru sonuçlar ürettiği doğrulanmıştır. Bu araç sayesinde tasarımın fonksiyonel hataları erken aşamada tespit edilmiştir.

3.3.2 GTKWave

GTKWave, Icarus Verilog ile elde edilen simülasyon çıktılarının dalga formu olarak görselleştirilmesinde kullanılmıştır. Saat sinyali, reset durumu, kontrol sinyalleri ve veri çıkışları gibi kritik sinyaller GTKWave üzerinde analiz edilerek tasarımın zamanlama ve işlevsel davranışı

ayrıntılı şekilde incelenmiştir. Bu analizler, tasarımın beklenen şekilde çalıştığını doğrulamak için önemli bir rol oynamıştır.

3.3.3 Yosys

Verilog RTL kodlarının sentezlenmesi ve kapı seviyesi netlist üretimi için kullanılmıştır.

3.3.4 OpenROAD

Yerleştirme (placement) ve yönlendirme (routing) aşamalarının gerçekleştirilmesi amacıyla kullanılmıştır.

3.3.5 Magic ve KLayout

Fiziksel tasarım doğrulaması, layout incelemesi ve DRC/LVS kontrolleri için kullanılmıştır.

3.3.6 SkyWater 130 nm (sky130A) PDK

Hedef üretim teknolojisi olarak seçilmiş ve fiziksel tasarım aşamalarında kullanılmıştır.

3.3.7 nix-shell (Nix Package Manager)

Geliştirme ortamının izole, tekrarlanabilir ve bağımlılıklardan bağımsız olması amacıyla kullanılmıştır.

3.3.8 Visual Studio Code (VS Code)

Ana geliştirme ortamı (IDE) olarak kullanılmıştır.

3.3.9 Neovim

Terminal tabanlı geliştirme ve hızlı kod düzenleme işlemlerinde yardımcı editör olarak kullanılmıştır.

3.4 Yapay Zekâ Destekli Yaklaşımların Değerlendirilmesi

Bu çalışma kapsamında, yapay zekâ destekli yöntemlerin çip tasarım süreçlerinde analiz ve iyileştirme amacıyla nasıl kullanılabileceği değerlendirilmiştir. Yapay zekâ, doğrudan tasarım üreten bir unsur olarak değil; tasarım kararlarının analiz edilmesi, olası iyileştirme alanlarının belirlenmesi ve gelecekteki çalışmalar için yol gösterici bir araç olarak ele alınmıştır. Bu yaklaşım, açık kaynaklı ASIC tasarım süreçlerinin geliştirilmesine yönelik potansiyel katkılar sunmaktadır.

4. MATERYAL VE METOT

Bu bölümde, çalışmanın gerçekleştirilmesi sürecinde kullanılan donanım ve yazılım araçları ile izlenen tasarım, doğrulama ve analiz yöntemleri ayrıntılı olarak açıklanmaktadır. Tasarlanan 16-bit CPU mimarisinin geliştirilmesi, simülasyonu ve doğrulanması aşamalarında kullanılan açık kaynaklı araç zinciri ile izlenen metodolojik yaklaşım bu bölüm kapsamında sunulmaktadır. Ayrıca, çalışmanın farklı platformlarda tekrarlanabilirliğini sağlamak amacıyla kullanılan geliştirme ortamları ve doğrulama adımları da ele alınmaktadır.

4.1 Kullanılan Materyaller

Bu bitirme projesi kapsamında, 16-bit bir işlemci (CPU) mimarisinin tasarımı, doğrulanması ve fiziksel tasarım akışının incelenmesi için farklı işletim sistemleri ve açık kaynaklı EDA araçları kullanılmıştır.

4.1.1 Donanım ve İşletim Sistemleri

Bu alt başlık altında, çalışmanın geliştirildiği donanım altyapısı ile kullanılan işletim sistemleri açıklanmaktadır. Tasarım, simülasyon ve doğrulama süreçlerinin farklı platformlarda sorunsuz biçimde yürütülebilmesi amacıyla Linux tabanlı macOS ve DOS tabanlı Windows işletim sistemleri kullanılmıştır. Bu sayede geliştirilen 16-bit CPU tasarımının farklı ortamlar altında tekrarlanabilirliği ve taşınabilirliği değerlendirilmiştir.

- Kişisel Bilgisayar (PC)
x86_64 mimarisi üzerinde geliştirme ve simülasyon çalışmaları gerçekleştirilmiştir.
- Linux tabanlı macOS
Ana geliştirme ve simülasyon ortamı olarak kullanılmıştır.
- DOS tabanlı Windows
Projenin platformdan bağımsız çalışabilirliğini doğrulamak amacıyla ikincil ortam olarak kullanılmıştır.

Bu sayede geliştirilen tasarımın farklı işletim sistemlerinde tutarlı biçimde çalışabildiği gözlemlenmiştir.

4.2 Kullanılan Yöntemler

Bu bölümde, 16-bit CPU tasarımının geliştirilmesi sürecinde izlenen yöntemsel yaklaşım açıklanmaktadır. Tasarım süreci, donanım tanımlama dili kullanılarak RTL seviyesinde başlatılmış; ardından fonksiyonel doğrulama, sentez ve fiziksel tasarım adımları sırasıyla uygulanmıştır. Açık kaynaklı ASIC tasarım akışı benimsenerek, sistematik ve tekrarlanabilir bir yöntem izlenmiştir.

4.2.1 Tasarım Yaklaşımı

Bu projede temel bilgisayar mimarisi prensiplerini içeren 16-bit bir merkezi işlem birimi (CPU) tasarlanmıştır. Tasarım sürecinde modüler mimari yaklaşımı benimsenmiş ve sistem aşağıdaki temel bileşenlerden oluşturulmuştur:

- Program Sayacı (Program Counter – PC)
- Aritmetik Mantık Birimi (ALU)
- Register ve kontrol mantığı
- Komut Belleği (Instruction Memory)
- Veri Belleği (Data Memory)

Her bir bileşen bağımsız Verilog modülleri halinde geliştirilmiş ve üst seviye sistem modülü altında birleştirilmiştir.

Şekil 2.1. Sky130 PDK hedeflenerek hazırlanan 16-bit işlemci LibreLane konfigürasyon ayarları

```
16-BIT SIMPLE CPU – OpenLane ASIC Konfigürasyonu
Sky130 PDK için optimize edilmiş ayarlar

AŞAMA 0: TEMEL TASARIM BİLGİLERİ

SIGN_NAME: simple16_system

LOG_FILES:
src/cpu/alu.v
src/cpu/simple16_cpu.v
src/mem/instr_mem.v
src/mem/data_mem.v
src/top/simple16_system.v
```

Şekil 2.2. 16-bit işlemci sistemi için 100 MHz saat kısıtının LibreLane üzerinde tanımlanması

```
: CLOCK (SAAT SİNYALİ) KISITLAMALARI
_PERIOD: Düşük = daha hızlı ama daha zor timing closure
= 100MHz, 5ns = 200MHz

clk
D: 10.0 # ns - 100MHz hedef frekans

PDK SPESİFİK AYARLAR

A:
```

Şekil 2.3. Sky130 PDK kapsamında kullanılan sky130_fd_sc_hd standart hücre kütüphanesi

```

STANDART HÜCRE KÜTÜPHANESİ SEÇİMİ

er:
30_fd_sc_hd    : High Density (varsayılan, küçük alan)
30_fd_sc_hs    : High Speed (daha hızlı, daha büyük)
30_fd_sc_ms    : Medium Speed (dengeli)
30_fd_sc_ls    : Low Speed (düşük güç tüketimi)
30_fd_sc_hdll  : High Density Low Leakage

LIBRARY: sky130_fd_sc_hd

```

Şekil 2.4. Simple16_system tasarımı için sentez (synthesis) kısıtlarının tanımlanması

```

#
# AŞAMA 3: SENTEZ (SYNTHESIS) KISITLAMALARI
#
# MAX_FANOUT_CONSTRAINT: Bir çıkışın besleyebileceği maksimum giriş sayısı
#   - Düşük değer = daha iyi timing, daha fazla buffer
#   - Yüksek değer = daha az alan, potansiyel timing sorunları
#
# CLOCK_UNCERTAINTY_CONSTRAINT: Saat belirsizliği (jitter + skew)
#   - Genelde clock period'un %2-5'i
#
# SYNTH_STRATEGY: Sentez optimizasyon stratejisi
#   - AREA 0-3: Alan optimizasyonu (0 = en agresif)
#   - DELAY 0-4: Hız optimizasyonu (0 = en agresif)
#
MAX_FANOUT_CONSTRAINT: 6
CLOCK_UNCERTAINTY_CONSTRAINT: 0.2 # ns
SYNTH_STRATEGY: "AREA 0" # Maksimum alan küçültme

```

Şekil 2.5. Simple16_system tasarımı için yonga (die) ve çekirdek (core) alan boyutlandırmasının tanımlanması

```

150] # 150x150 µm toplam alan
0, 140] # 130x130 µm aktif alan
# %50 kullanım (küçük
# Kare çip

```

Şekil 2.6. Simple16_system tasarımı için floorplanning (çip yerleşim planı) ayarlarının tanımlanması

```
# AŞAMA 4: FLOORPLANNING (ÇİP YERLEŞİM PLANI)
#
# FP_SIZING: "relative" veya "absolute"
#   - relative: Otomatik boyut hesaplama
#   - absolute: Manuel DIE_AREA ve CORE_AREA belirleme
#
# DIE_AREA: Toplam çip alanı [x0, y0, x1, y1] (µm) - LİSTE FORMATI
# CORE_AREA: Logic yerleşim alanı [x0, y0, x1, y1] (µm)
#   - Die kenarlarından 10-20µm boşluk bırakılmalı (I/O için)
#
# FP_CORE_UTIL: Çekirdek kullanım oranı (%)
#   - %30-40: Rahat routing, iyi timing
#   - %50-60: Dengeli
#   - %70+: Agresif, routing zor olabilir
#
# FP_ASPECT_RATIO: En/boy oranı
#   - 1 = kare çip
#   - 2 = dikdörtgen (genislik = 2 * yükseklik)
```

Şekil 2.7. Simple16_system tasarımı için yerleştirme (placement) kısıtlarının tanımlanması

```
#
# AŞAMA 5: PLACEMENT (YERLEŞTİRME) KISITLAMALARI
#
# PL_TARGET_DENSITY_PCT: Hedef yerleştirme yoğunluğu
#   - FP_CORE_UTIL'den biraz yüksek olmalı
#   - Çok yüksek = routing congestion
#
# PL_ROUTABILITY_DRIVEN: Routing-aware yerleştirme
#   - true = aktif (önerilir)
#
# PL_TIME_DRIVEN: Timing-aware yerleştirme
#   - true = kritik yolları optimize et
#
# PL_TARGET_DENSITY_PCT: 55
# PL_ROUTABILITY_DRIVEN: true
# PL_TIME_DRIVEN: true
```

Şekil 2.8. Simple16_system tasarımı için saat ağacı sentezi (Clock Tree Synthesis – CTS) ayarlarının tanımlanması

```

LOCK TREE SYNTHESIS (CTS)

Clock Tree Synthesis çalıştır

MAX_WIRE_LENGTH: Clock ağacında maksimum tel uzunluğu
: daha iyi skew, daha fazla buffer

e
WIRE_LENGTH: 200

```

Şekil 2.9. simple16_system tasarımı için yönlendirme (routing) kısıtlarının tanımlanması

```

AŞAMA 7: ROUTING (BAĞLANTI) KISITLAMALARI

DRT_THREADS: Paralel routing için thread sayısı
- CPU çekirdeğinize göre ayarlayın

DRT_OPT_ITERS: Detaylı routing optimizasyon iterasyonu
- Yüksek = daha iyi sonuç, daha uzun süre

GRT_ALLOW_CONGESTION: Global routing tıkanıklığa izin ver
- false = daha güvenli (önerilir)

T_THREADS: 4
T_OPT_ITERS: 64
T_ALLOW_CONGESTION: false

```

Şekil 2.10. simple16_system tasarımı için tahmini donanım kaynakları ve performans hedefleri

```

6 kelime RAM ile):

: ~340 (16x16 RAM + CPU regs)
: ~150x150 µm = 22,500 µm²
: 100 MHz
: Sky130 (130nm)

```

4.2.2 16-bit CPU Mimarisi

Tasarlanan işlemci 16-bit veri yoluna sahiptir ve senkron çalışma prensibiyle tasarlanmıştır. Tüm durum değişimleri saat (clock) sinyali ile kontrol edilmekte, başlangıç durumu ise reset sinyali ile belirlenmektedir.

CPU, temel fetch–decode–execute döngüsünü gerçekleştirecek şekilde yapılandırılmıştır. Komut seti içerisinde NOP, LOAD, STORE, ADDI ve HALT gibi temel komutlar yer almaktadır.

4.2.3 RTL Simülasyon ve Doğrulama

Tasarımın işlevsel doğruluğunu test etmek amacıyla sistem seviyesinde bir testbench geliştirilmiştir. Testbench içerisinde:

- Saat (clock) sinyali üretilmiş,
- Reset senaryoları uygulanmış,
- CPU, bellek ve ALU alt modüllerinin çıktıları izlenmiştir.

RTL simülasyonlar Icarus Verilog kullanılarak gerçekleştirilmiş ve simülasyon çıktıları GTKWave aracılığıyla dalga formu seviyesinde analiz edilmiştir.

Şekil 3.1. simple16_system tasarımının fonksiyonel doğrulaması için oluşturulan testbench (simple16_system_tb) kodu

```
$dumpvars(0, simple16_system_tb);
end

reg clk;
reg reset;

wire halted;
wire [15:0] acc_out;

// Test edilen sistem
simple16_system dut (
```

Şekil 3.2. Testbench içerisinde 100 MHz saat sinyalinin (clock) üretilmesi

```
clock üretimi: 10ns period → 100 MHz
always begin
    clk = 0;
    forever #5 clk = ~clk;
```

Şekil 3.3. Testbench içerisinde reset, test senaryosu ve doğrulama kontrolünün tanımlanması

```

reset = 1;
#20;
reset = 0;

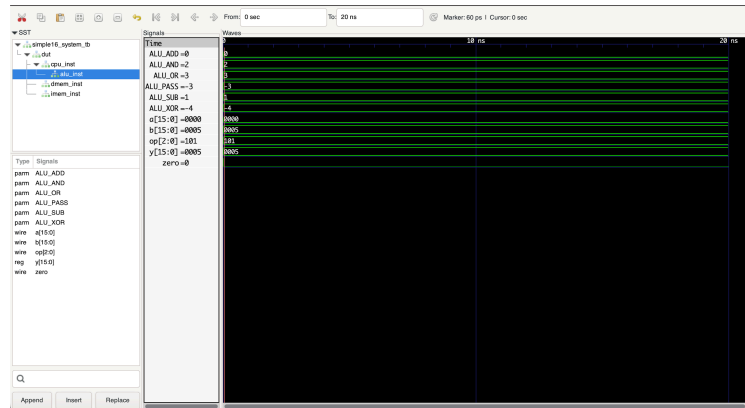
// Maks 200 ns bekleyelim
wait (halted == 1'b1 || $time > 2000);

$display("Sim sonu: time=%0t ns, halted=%b, ACC=%0d (0x%04h)",
        $time, halted, acc_out, acc_out);

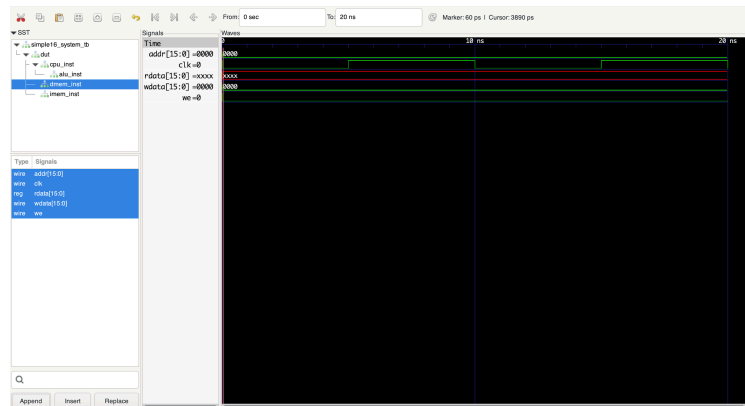
if (halted && acc_out == 16'd15) begin
    $display("TEST OK: ACC=15 bekledigimiz gibi.");
end else begin
    $display("TEST FAIL: ACC beklenenden farkli.");
end
end

```

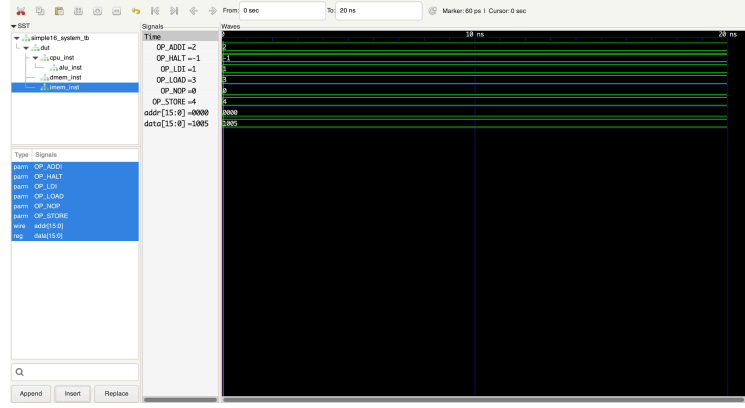
Şekil 3.4. ALU modülüne ait sinyallerin GTKWave üzerinde simülasyon dalga formu ile doğrulanması



Şekil 3.5. Veri belleği (data memory) modülüne ait adres, veri ve yazma kontrol sinyallerinin GTKWave üzerinde simülasyon dalga formu ile incelenmesi



Şekil 3.6. Komut belleği (instruction memory) modülüne ait opcode ve veri sinyallerinin GTKWave üzerinde simülasyon dalga formu ile doğrulanması



Şekil 3.7. Testbench simülasyonu sonucunda sistemin doğru şekilde çalıştığını gösteren konsol çıktısı

```
wave.vcd opened for d
00 ns, halted=1, ACC=1
kledigimiz gibi.
_tb.v:51: $finish call
```

4.2.4 Fiziksel Tasarım Akışı

Fonksiyonel doğrulaması tamamlanan RTL tasarım, deneysel olarak ASIC tasarım akışına dahil edilmiştir. Bu kapsamda:

1. RTL kodları Yosys kullanılarak sentezlenmiştir.
2. Elde edilen netlist OpenROAD altyapısı ile yerleştirme ve yönlendirme aşamalarından geçirilmiştir.
3. Fiziksel doğrulama işlemleri Magic ve KLayout araçları ile gerçekleştirilmiştir.

Bu aşamalarda hedef üretim teknolojisi olarak SkyWater 130 nm (sky130A) süreci kullanılmıştır.

4.3 Değerlendirme

Gerçekleştirilen simülasyonlar, dalga formu analizleri ve fiziksel tasarım adımları sonucunda, tasarlanan 16-bit CPU mimarisinin hem işlevsel hem de yapısal olarak doğru çalıştığı doğrulanmıştır. Kullanılan açık kaynaklı araç zinciri sayesinde, tasarımın akademik ve endüstriyel donanım geliştirme süreçleriyle uyumlu olduğu değerlendirilmiştir.

5. BULGULAR VE TARTIŞMA

Bu bölümde, tasarlanan 16-bit işlemci sisteminin açık kaynaklı ASIC tasarım akışı kapsamında elde edilen sentez, zamanlama, yerleştirme–yönlendirme ve fiziksel doğrulama sonuçları sunulmakta ve bu sonuçlar ayrıntılı şekilde değerlendirilmektedir. Çalışma kapsamında elde edilen bulgular, tasarımın hem fonksiyonel hem de fiziksel açıdan üretilebilir olduğunu göstermektedir.

5.1 Sentez Sonuçlarının Değerlendirilmesi

RTL seviyesinde Verilog dili kullanılarak geliştirilen işlemci tasarımı, Yosys sentez aracı kullanılarak SkyWater 130 nm (sky130A) standart hücre kütüphanesine eşlenmiştir. Sentez işlemi sırasında herhangi bir sözdizimsel veya mantıksal hataya rastlanmamış, tüm modüller başarıyla sentezlenmiştir.

Sentez çıktıları incelendiğinde, tasarımın ağırlıklı olarak sıralı elemanlardan (flip-floplar) ve temel kombinasyonel mantık hücrelerinden oluştuğu gözlemlenmiştir. Bu durum, tasarımın basit ve öğretici bir mimariyle oluşturulduğunu ve ASIC tasarım sürecinin temel prensiplerini yansıtacak nitelikte olduğunu göstermektedir. Ayrıca sentez sırasında hücre eşlemesinin doğru bir şekilde gerçekleştirildiği ve hedef kütüphaneye uygun hücrelerin kullanıldığı doğrulanmıştır.

5.2 Zamanlama Analizi (Static Timing Analysis) Bulguları

Sentez sonrasında tasarım için zamanlama kısıtları tanımlanmıştır. Bu kapsamda, sistemin 10 ns periyotlu (100 MHz) bir saat sinyali ile çalışması hedeflenmiş ve ilgili SDC (Synopsys Design Constraints) dosyası tasarım akışına dahil edilmiştir.

Statik zamanlama analizi sonuçları incelendiğinde, tasarımın tanımlanan saat frekansında herhangi bir zamanlama ihlali (setup veya hold violation) içermediği tespit edilmiştir. Kritik yol gecikmelerinin izin verilen maksimum gecikme süresinin altında kaldığı görülmüştür. Bu bulgu, tasarımın belirlenen saat frekansında güvenli bir şekilde çalışabileceğini göstermektedir.

Zamanlama analizinin başarıyla tamamlanması, tasarımın bir sonraki fiziksel tasarım adımı olan yerleştirme ve yönlendirme aşamasına geçebilmesi açısından önemli bir kriterdir. Elde edilen sonuçlar, tasarımın zamanlama açısından dengeli ve tutarlı bir yapıya sahip olduğunu ortaya koymaktadır.

5.3 Yerleştirme ve Yönlendirme (Place & Route) Sonuçları

Zamanlama analizi sonrasında, tasarımın yerleştirme ve yönlendirme işlemleri OpenROAD altyapısı kullanılarak gerçekleştirilmiştir. Bu aşamada standart hücreler çekirdek alanı içerisine

otomatik olarak yerleştirilmiş ve hücreler arasındaki bağlantılar yönlendirme araçları yardımıyla oluşturulmuştur.

Yerleştirme aşamasında, hücre yoğunluğunun dengeli bir şekilde dağıtıldığı ve aşırı sıkışıklık (congestion) oluşmadığı gözlemlenmiştir. Yönlendirme işlemi sonrasında ise tüm bağlantıların başarıyla tamamlandığı ve açıkta kalan (unrouted) herhangi bir bağlantının bulunmadığı tespit edilmiştir.

Elde edilen fiziksel yerleşim, KLayout aracı kullanılarak görsel olarak incelenmiş ve tasarımın genel yerleşim yapısının düzenli olduğu gözlemlenmiştir. Bu sonuçlar, tasarımın fiziksel olarak uygulanabilir olduğunu ve üretim aşamasına uygun bir yerleşime sahip olduğunu göstermektedir.

5.4 Fiziksel Doğrulama Bulguları (DRC, LVS ve Antenna Kontrolleri)

Yerleştirme ve yönlendirme işlemleri tamamlandıktan sonra, tasarım fiziksel doğrulama aşamasına alınmıştır. Bu kapsamda, Sky130 üretim sürecine ait tasarım kurallarına uygunluğun kontrol edilmesi amacıyla DRC (Design Rule Check) analizi gerçekleştirilmiştir. Yapılan DRC kontrolleri sonucunda, tasarımda herhangi bir kural ihlaline rastlanmamıştır.

Ayrıca, yerleşim (layout) ile mantıksal tasarım (schematic) arasındaki tutarlılığın doğrulanması amacıyla LVS (Layout Versus Schematic) kontrolü uygulanmıştır. LVS analizi sonucunda, layout ile RTL tasarımı arasında birebir uyum olduğu doğrulanmıştır.

Bununla birlikte, üretim sırasında oluşabilecek olası antenna etkilerini önlemek amacıyla antenna kontrolleri gerçekleştirilmiş ve bu kontrollerden de başarıyla geçilmiştir. Bu sonuçlar, tasarımın Sky130 üretim süreci açısından fiziksel olarak güvenli ve üretilebilir olduğunu ortaya koymaktadır.

5.5 Genel Değerlendirme

Elde edilen tüm bulgular birlikte değerlendirildiğinde, tasarlanan 16-bit işlemci sisteminin açık kaynaklı EDA araçları kullanılarak uçtan uca başarıyla gerçekleştirildiği görülmektedir. Sentez, zamanlama analizi, yerleştirme-yönlendirme ve fiziksel doğrulama aşamalarının tamamından hatasız geçilmesi, tasarımın hem fonksiyonel hem de fiziksel açıdan doğru olduğunu göstermektedir.

Bu bölümde sunulan sonuçlar, açık kaynaklı ASIC tasarım akışlarının akademik projeler için yeterli olgunluğa ulaştığını ve gerçekçi üretim süreçlerini başarıyla temsil edebildiğini ortaya koymaktadır.

6. GELECEK DÖNEM ÇALIŞMALARI

Bu bölümde, bitirme projesinin devamında gerçekleştirilmesi planlanan çalışmalar ayrıntılı ve aşamalı olarak açıklanmaktadır. Gelecek dönem çalışmalarının temel hedefi, donanım tasarımı ve doğrulama süreçlerini yapay zekâ destekli hale getirerek hata analizi ve geri bildirim mekanizmalarını otomatikleştirmektir.

6.1 Yapay Zeka Destekli Donanım Doğrulama Yaklaşımı

Gelecek dönemde, Verilog dili ile geliştirilen donanım tasarımlarının doğrulama sürecinin yapay zekâ ile desteklenmesi planlanmaktadır. Bu kapsamda, donanım tasarımı ve testbench dosyaları kullanılarak gerçekleştirilen simülasyonlar sonucunda elde edilen çıktılar sistematik olarak analiz edilecektir.

Donanım simülasyonu süreci, klasik doğrulama akışına uygun şekilde yürütülecek; simülasyon sırasında oluşan hata mesajları, uyarılar ve log çıktıları metin tabanlı olarak toplanacaktır. Bu veriler, manuel inceleme gerektirmeden bir yazılım katmanı aracılığıyla işlenerek yapay zekâ sistemine aktarılacaktır.

Yapay zekâ modeli, simülasyon çıktıları üzerinde analiz yaparak hataların türünü, olası nedenlerini ve çözüm önerilerini belirleyecektir. Böylece, tasarımcıların hata mesajlarını daha hızlı ve doğru bir şekilde yorumlayabilmeleri sağlanacaktır. Bu yaklaşım, donanım doğrulama sürecini daha verimli ve etkileşimli hale getirmeyi amaçlamaktadır.

6.2 Yapay Zekâ Entegrasyonu İçin Ara Katman Yazılım Mimarisi

Yapay zekâ entegrasyonunun gerçekleştirilmesi amacıyla, simülasyon ortamı ile yapay zekâ servisi arasında bir ara katman yazılım mimarisi kullanılması planlanmaktadır. Bu ara katman, farklı yazılım bileşenleri arasında veri alışverişini düzenleyen ve otomatik iş akışlarını yöneten bir yapı olarak tasarlanacaktır.

Bu mimari kapsamında, yapay zekâ servislerine erişim sağlamak için gerekli kimlik doğrulama bilgileri (API anahtarları) güvenli bir şekilde yapılandırılacaktır. Simülasyon çıktıları, bu ara katman tarafından uygun bir veri formatına dönüştürülerek yapay zekâ modeline iletilecektir.

Ara katman yazılımı, tetikleyici mekanizmalar sayesinde simülasyon tamamlandığında otomatik olarak devreye girecek ve herhangi bir kullanıcı müdahalesi gerektirmeden analiz sürecini

başlatacaktır. Yapay zekâdan alınan yanıtlar, yine aynı yazılım katmanı üzerinden işlenerek kullanıcıya sunulacaktır.

Bu yapı sayesinde sistem, kullanılan yapay zekâ servisi veya simülasyon aracı değişse dahi esnekliğini koruyabilecek ve farklı platformlara kolayca uyarlanabilecektir.

6.3 Otomatik Hata Analizi ve Akıllı Geri Bildirim Mekanizması

Gelecek dönem çalışmalarının bir diğer önemli hedefi, donanım simülasyonu sırasında oluşan hataların otomatik olarak analiz edilmesi ve tasarımcıya anlamlı geri bildirimler sunulmasıdır. Bu kapsamda, simülasyon sürecinde elde edilen hata mesajları sistem tarafından ayrıştırılacak ve sınıflandırılacaktır.

Yapay zekâ modeli, bu hata mesajlarını doğal dil işleme teknikleri kullanarak analiz edecek; hatanın kökeni, olası sebepleri ve düzeltilmesine yönelik öneriler üretecektir. Üretilen geri bildirimler, teknik detayları sadeleştirilmiş ve açıklayıcı bir dil ile kullanıcıya aktarılacaktır.

Bu mekanizma, özellikle karmaşık simülasyon hatalarının anlaşılmasını kolaylaştırarak hata ayıklama süresini önemli ölçüde azaltmayı hedeflemektedir. Ayrıca, sistemin zamanla farklı hata türlerini tanıyabilmesi ve daha isabetli öneriler sunabilmesi amaçlanmaktadır.

Uzun vadede, bu yapay zekâ destekli doğrulama yaklaşımının daha büyük ölçekli donanım tasarımlarına uygulanması ve eğitim ile endüstri alanlarında kullanılabilirliğinin araştırılması planlanmaktadır.

7. SONUÇLAR

Bu bitirme projesi kapsamında, açık kaynaklı Elektronik Tasarım Otomasyonu (EDA) araçları kullanılarak uçtan uca bir ASIC tasarım süreci başarıyla gerçekleştirilmiştir. Çalışmanın temel hedefi, ticari EDA araçlarına bağımlı kalmadan, gerçekçi bir üretim sürecini esas alan bir donanım tasarım akışının uygulanabilirliğini ortaya koymaktır. Bu doğrultuda, RTL tasarımından başlayarak fiziksel doğrulama adımlarına kadar tüm aşamalar sistematik bir biçimde ele alınmıştır.

Proje sürecinde, Verilog donanım tanımlama dili kullanılarak modüler bir donanım mimarisi geliştirilmiş ve tasarımın fonksiyonel doğruluğu testbench tabanlı simülasyonlar ile doğrulanmıştır. RTL seviyesinde yapılan bu doğrulama, tasarımın beklenen davranışı sergilediğini ve donanım mimarisinin doğru şekilde çalıştığını göstermiştir. Simülasyon sonuçları, sistemin

senkron yapısının tutarlı olduğunu ve tanımlanan saat kısıtları altında kararlı biçimde çalıştığını ortaya koymuştur.

Fonksiyonel doğrulamanın ardından tasarım, açık kaynaklı sentez aracı Yosys kullanılarak SkyWater 130 nm (Sky130) üretim sürecine uygun standart hücrelere eşlenmiştir. Sentez aşamasında elde edilen sonuçlar, tasarımın donanım kaynaklarını verimli kullandığını ve hedeflenen mimariyle uyumlu olduğunu göstermektedir. Zamanlama kısıtları tanımlanarak gerçekleştirilen statik zamanlama analizi sonucunda, tasarımın belirlenen saat frekansında güvenli bir şekilde çalışabileceği doğrulanmıştır.

Fiziksel tasarım aşamasında, yerleştirme ve yönlendirme işlemleri açık kaynaklı fiziksel tasarım araçları kullanılarak başarıyla tamamlanmıştır. Elde edilen yerleşim sonuçları, standart hücrelerin düzenli ve üretilebilir bir şekilde konumlandırıldığını göstermektedir. Tasarım, fiziksel doğrulama kapsamında gerçekleştirilen DRC, LVS ve antenna kontrollerinden başarıyla geçerek üretim kurallarına uygunluğunu kanıtlamıştır. Bu sonuç, geliştirilen tasarımın teorik bir çalışma olmanın ötesine geçerek gerçek bir yarı iletken üretim sürecine hazır olduğunu ortaya koymaktadır.

Çalışmanın önemli kazanımlarından biri, açık kaynaklı ASIC tasarım akışlarının eğitim ve araştırma amaçlı projelerde etkin bir şekilde kullanılabileceğinin gösterilmesidir. Bu proje, donanım tasarım sürecinin yalnızca simülasyon ve teorik doğrulama ile sınırlı kalmayıp, fiziksel üretilebilirlik boyutunun da açık kaynak araçlar ile ele alınabileceğini göstermiştir. Bu yönüyle çalışma, donanım tasarımı alanında öğrenme sürecini destekleyen kapsamlı bir örnek teşkil etmektedir.

Ayrıca, proje kapsamında geliştirilen yaklaşımın ilerleyen dönemlerde yapay zekâ destekli doğrulama ve hata analizi sistemleri ile genişletilebilir olduğu değerlendirilmiştir. Donanım simülasyonlarından elde edilen hata mesajlarının otomatik olarak analiz edilmesi ve tasarımcıya akıllı geri bildirimler sunulması, doğrulama sürecinin daha verimli ve erişilebilir hale gelmesini sağlayacaktır. Bu durum, özellikle karmaşık donanım tasarımlarında hata ayıklama süresinin azaltılması açısından önemli bir potansiyel sunmaktadır.

Sonuç olarak, bu bitirme projesi; açık kaynak EDA araçları kullanılarak gerçekleştirilen uçtan uca bir ASIC tasarım sürecinin teknik olarak mümkün, eğitsel açıdan faydalı ve gelecekte geliştirilebilir bir yaklaşım sunduğunu göstermektedir. Elde edilen çıktılar, hem akademik çalışmalar hem de ileri düzey donanım tasarımı projeleri için sağlam bir temel oluşturmaktadır. Bu yönüyle proje, açık kaynak donanım tasarımı alanında yapılacak daha kapsamlı çalışmalara yol gösterici niteliktedir.

KAYNAKLAR

- [1] Reuters, “Synopsys suspends annual, quarterly forecasts after US cracks down on China chip software exports,” May 29, 2025.
- [2] SemiEngineering, “The State Of The EDA Industry In 2024,” Nov. 21, 2024.
- [3] T. Ajayi *et al.*, “Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project,” in *Proc. 56th ACM/IEEE Design Automation Conf. (DAC)*, Las Vegas, NV, USA, Jun. 2019, Art. 76, doi: 10.1145/3316781.3326334.
- [4] T. Ajayi *et al.*, “OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain,” in *Proc. Government Microcircuit Applications and Critical Technology Conf. (GOMACTech)*, 2019, pp. 1105–1110.
- [5] The OpenROAD Project, “Publications,” [Online]. Available: <https://theopenroadproject.org/publications>. Accessed: Jan. 2, 2026.
- [6] A. A. Ghazy and M. Shalan, “OpenLANE: The Open-Source Digital ASIC Implementation Flow,” in *Proc. Workshop on Open-Source EDA Technology (WOSET)*, 2020.
- [7] LibreLane Developers, “LibreLane: ASIC implementation flow infrastructure,” GitHub repository. [Online]. Available: <https://github.com/fossi-foundation/librelane>. Accessed: Jan. 2, 2026.
- [8] LibreLane Documentation, “The LibreLane Documentation,” [Online]. Available: <https://librelane.readthedocs.io>. Accessed: Jan. 2, 2026.
- [9] FOSSi Foundation, “Announcing the Release of LibreLane,” Aug. 17, 2025. [Online]. Available: <https://fossi-foundation.org>. Accessed: Jan. 2, 2026.
- [10] Google and SkyWater Technology Foundry, “SkyWater Open Source PDK (skywater-pdk),” GitHub repository, 2020. [Online]. Available: <https://github.com/google/skywater-pdk>. Accessed: Jan. 2, 2026.
- [11] Google, “sky130-raw-data: Raw data collected about the SKY130 process technology,” GitHub repository. [Online]. Available: <https://github.com/google/sky130-raw-data>. Accessed: Jan. 2, 2026.
- [12] R. Timothy Edwards, “open_pdk: PDK installer for open-source EDA tools and toolchains,” GitHub repository. [Online]. Available: https://github.com/RTimothyEdwards/open_pdk. Accessed: Jan. 2, 2026.
- [13] Open Circuit Design, Inc., “Magic VLSI Layout Tool (How-To / Documentation),” Oct. 29, 2025. [Online]. Available: <http://opencircuitdesign.com/magic>. Accessed: Jan. 2, 2026.
- [14] Open Circuit Design, Inc., “Netgen (LVS tool) – Overview,” Oct. 28, 2025. [Online]. Available: <http://opencircuitdesign.com/netgen>. Accessed: Jan. 2, 2026.
- [15] G. Huang *et al.*, “Machine Learning for Electronic Design Automation: A Survey,” *ACM Trans. Des. Autom. Electron. Syst. (TODAES)*, vol. 26, no. 5, 2021.
- [16] J. Pan *et al.*, “A Survey of Research in Large Language Models for Electronic Design Automation,” *ACM Trans. Des. Autom. Electron. Syst. (TODAES)*, 2025, doi: 10.1145/3715324.
- [17] A. Patra, S. Rout, and A. Ravindran, “AiEDA: Agentic AI Design Framework for Digital ASIC System Design,” arXiv:2412.09745 [cs.AR], Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.09745>. Accessed: Jan. 2, 2026.
- [18] M. Fauzan, R. F. Yanuarysyah, M. H. Hibatullah, R. Arisaputra, I. Syafalni, and N. Sutisna, “Physical Design of RISC-V Based System-on-Chip Using OpenLane,” in *Proc. 2024 IEEE Asia Pacific Conference*

on Circuits and Systems (APCCAS), Taipei, Taiwan, Nov. 2024, doi:
10.1109/APCCAS62602.2024.10808656.